
Análisis de distintas formulaciones de los problemas TSP y STSP

Daniel Lugaresi Palomares, Iker Rodríguez Rodríguez

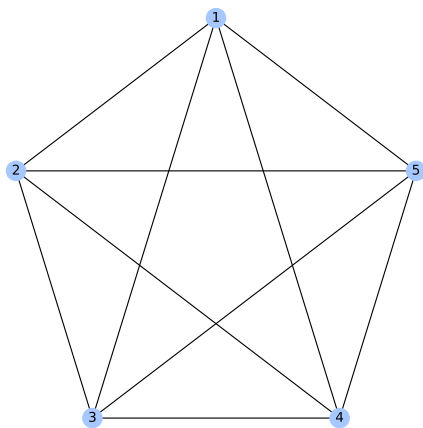
7 de junio de 2026

1. Planteamiento del trabajo

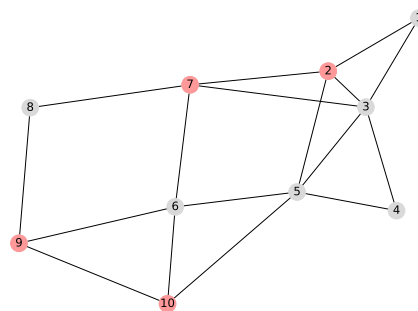
El *problema del viajante* (**TSP**) es ampliamente utilizado para modelizar situaciones como, por ejemplo, repartos de paquetería. Podemos pensar en un camión que sale de un almacén con n paquetes que debe entregar, cada uno a un cliente diferente. Entonces, el repartidor puede visitar a cada cliente en el orden que quiera, con un coste fijo determinado, que podría depender de la distancia a la vivienda de dicho cliente. En esta situación, podemos pensar en un grafo *simple, completo* y *sin lazos* $\mathcal{G} = (N, G)$, es decir, todos los nodos (clientes) están conectados por una única arista (conexiones), y no existen aristas del tipo (i, i) . Además, cada arista tiene un coste asociado a viajar por ella, y nuestro objetivo sería seleccionar la ruta óptima para minimizar el coste total del viaje. En general, las formulaciones que existen de este problema asumen que cada arista se puede utilizar una sola vez, lo cual tiene sentido, pues una vez visitamos a un cliente no tenemos razón para volver a su domicilio.

Sin embargo, imaginemos que la empresa de paquetería tiene una serie de pedidos urgentes que debe entregar ese mismo día, obligatoriamente; y además supongamos que hay varias calles cortadas, por las que no se puede pasar. En este caso, el repartidor tal vez necesite visitar algunos clientes no urgentes para poder desplazarse y llegar a los clientes urgentes, o tal vez hacerlo resulte en un coste menor que ir directamente al domicilio de los mismos.

Además, podría ser necesario recorrer algún camino más de una vez, por ejemplo si un cliente vive en una calle cortada por las obras. Este tipo de situaciones se modelan a través de la variación del problema del viajante conocida como *Problema del viajante de Steiner* (**STSP**), que parte de un grafo *simple* y sin *lazos* pero no necesariamente completo, $\mathcal{G} = (N, G)$. Además, se añade un subconjunto de nodos requeridos por los que debe pasar cada ruta factible, $N_R \subseteq N$. Esta modelización sirve también para abordar otro tipo de problemas, como por ejemplo la inspección de subestaciones eléctricas. Podemos ver un ejemplo de los grafos asociados a cada problema en la Figura 1.



(a) Ejemplo de grafo completo que modeliza el problema del viajante (**TSP**). Tenemos 5 posibles clientes.



(b) Ejemplo de grafo no completo que modeliza el problema de Steiner (**STSP**). Tenemos 10 clientes con 4 entregas urgentes (en rojo), y no podemos ir directamente a cualquiera.

Figura 1: Ejemplo ilustrativo de los problemas que consideramos en el trabajo.

En este trabajo, estudiaremos en diferentes formulaciones del problema del viajante original, y veremos cómo se utilizan como base para las distintas formulaciones del problema de Steiner. Después, realizaremos un estudio computacional que nos permitirá diferenciar las formulaciones según su rendimiento en distintas situaciones, para ambos problemas. En cuanto a la documentación de este trabajo, nos hemos basado en un *paper*, [1], que recopila distintas formulaciones para ambos problemas.

2. Definición de los problemas y sus formulaciones

2.1. Problema original (TSP)

Revisaremos cuatro formulaciones del *problema del viajante* original: la primera será la formulación de Dantzig-Fulkerson-Johnson, que será la más problemática, como veremos, por el rápido escalado del número de restricciones. Después, veremos tres reformulaciones con filosofías distintas: dos basadas en flujos, *Single-Commodity* y *Multicommodity*, y una última en la estratificación temporal de las variables, *Time-Staged*. Todos los resultados

teóricos mencionados en esta sección se pueden encontrar en la referencia [1].

Para comenzar, sea un grafo simple, completo y sin lazos $\mathcal{G} = (N, G)$, siendo N el conjunto de nodos, es decir, los distintos clientes; y G el conjunto de aristas o arcos, de la forma $(i, j) \in G \subset N \times N, i \neq j$. Denotamos $n = |N|$.

Cada arista tendrá asociado un coste c_{ij} , que puede ser entendido como tiempo de viaje, distancia, precio del transporte, etc. Será nuestro objetivo minimizar el coste total de la ruta elegida.

2.1.1. Formulación de Dantzig-Fulkerson-Johnson (TSP-DFJ)

Se formula de la siguiente manera:

Variables:	Datos:
$x_{ij} = \begin{cases} 1, & \text{la ruta va de } i \text{ a } j \\ 0, & \text{en otro caso.} \end{cases}$	$c_{ij}: \text{el coste de ir de } i \text{ a } j.$

Minimizar $\sum_{i \in N} \sum_{j \in N, j \neq i} c_{ij} x_{ij}$

sujeto a

$$\sum_{j \in N, j \neq i} x_{ij} = 1 \quad \forall i \in N, \quad (1)$$

$$\sum_{j \in N, j \neq i} x_{ji} = 1 \quad \forall i \in N, \quad (2)$$

$$\sum_{i \in S} \sum_{j \in S} x_{ij} \leq |S| - 1 \quad \forall S \subsetneq N, |S| > 1, \quad (3)$$

$$x_{ij} \in \{0, 1\} \quad i \neq j \in N \times N.$$

El significado de cada restricción es el siguiente:

- (1) Debemos salir de cada ciudad exactamente una vez.
- (2) Debemos entrar en cada ciudad exactamente una vez.
- (3) *Restricciones de eliminación de subtours*: no se pueden formar circuitos cerrados que no sean la solución final. Obligamos, de formarse uno, a que tenga que unirse algún nodo del circuito con otro que no esté en el circuito.

Los problemas de esta formulación provienen precisamente de la restricción (3), que permite que esté bien definida: al necesitar comprobar todos los distintos subconjuntos de nodos, el número de restricciones va a crecer de forma exponencial conforme aumentemos el tamaño de N .

Esta clase de formulaciones, donde el crecimiento del modelo no es polinómico, se conoce como formulaciones *no compactas*. Por contraposición, denominaremos *compacta* a cual-

quier formulación cuyo crecimiento —tanto en número de restricciones como en número de variables— al aumentar el tamaño sea polinómico. Será de nuestro interés encontrar formulaciones compactas del problema, ahorrando así memoria del ordenador y tiempo de resolución.

2.1.2. Formulación de flujo *Single-Commodity* (TSP-SCF)

Entre las formulaciones vistas en clase que cumplan ser *compactas* podemos destacar la formulación **TSP-SCF** —principalmente porque veremos más adelante una formulación que se inspira en esta para el problema general de Steiner—. Supongamos ahora que, en vez de movernos entre clientes, queremos minimizar el coste de mover una cierta cantidad de flujo por un sistema. Consideremos, entonces f_{ij} el flujo que transcurre por la arista de i a j . Buscamos repartir $n - 1$ unidades de flujo (que parten del nodo inicial) por el sistema, enviándolas desde el nodo inicial y dejando una unidad en cada uno de los nodos. La formulación se encuentra a continuación:

Variables:

Datos:

$$x_{ij} = \begin{cases} 1, & \text{la ruta va de } i \text{ a } j \\ 0, & \text{en otro caso.} \end{cases} \quad c_{ij}: \text{ el coste de mover flujo de } i \text{ a } j.$$

f_{ij} : flujo que se mueve de i a j

Sea ahora $e_1 = n - 1$ y $e_i = -1$ para todo $i \in N \setminus \{1\}$:

$$\begin{aligned} \text{Minimizar} \quad & \sum_{i \in N} \sum_{j \in N, j \neq i} c_{ij} x_{ij} \\ \text{sujeto a} \quad & \sum_{j \in N, j \neq i} x_{ij} = 1 \quad \forall i \in N, \quad (1) \\ & \sum_{j \in N, j \neq i} x_{ji} = 1 \quad \forall i \in N, \quad (2) \\ & \sum_{j \in N, j \neq i} f_{ij} - \sum_{j \in N, j \neq i} f_{ji} = e_i \quad i \in N, \quad (3) \\ & f_{ij} \leq (n - 1)x_{ij} \quad i \neq j \in N \times N. \quad (4) \\ & f_{ij} \geq 0, \quad i \neq j \in N \times N. \\ & x_{ij} \in \{0, 1\}, \quad i \neq j \in N \times N. \end{aligned}$$

Las restricciones (1) y (2) tienen el mismo significado visto en **TSP-DFJ**, mientras que las restricciones (4) quieren decir que no podemos enviar flujo por las aristas que no son usadas (*restricciones de enlace*). Por último, (3) son las *restricciones de flujo del sistema*, que fuerzan a que en cada nodo se extraiga una unidad de flujo —salvo en el nodo inicio donde el flujo entra al sistema—.

Esta formulación tiene muchas menos restricciones que **TSP-DFJ**, pero intercambia esta mejoría en la optimización de memoria con un *Lower Bound* de su problema relajado más pobre en comparación.

Introduciremos ahora dos nuevas formulaciones para el problema del viajante original, que será con las que más trabajemos.

2.1.3. Formulación de flujo *Multicommodity Flow* (TSP-MCF)

La idea de esta formulación es ligeramente distinta respecto a **TSP-SCF**: seguimos pensando en $n - 1$ unidades de flujo que salen de la primera ciudad, dejando una en cada nodo, pero ahora marcamos cada unidad de flujo según su destino previsto, el nodo k . Así, distinguimos qué destino tiene una unidad de flujo que pasa por una arista determinada. Es decir, ahora realizamos un seguimiento del camino realizado por cada unidad de flujo por separado. Tenemos la siguiente formulación:

Variables:

Datos:

$$x_{ij} = \begin{cases} 1, & \text{la ruta va de } i \text{ a } j \\ 0, & \text{en otro caso.} \end{cases} \quad c_{ij}: \text{ el coste de mover flujo de } i \text{ a } j.$$

f_{ij}^k : flujo que se mueve de i a j con destino al nodo k .

$$\begin{aligned} &\text{Minimizar} && \sum_{i \in N} \sum_{j \in N, j \neq i} c_{ij} x_{ij} \\ &\text{sujeto a} && \sum_{j \in N, j \neq i} x_{ij} = 1 && \forall i \in N, && (1) \\ &&& \sum_{j \in N, j \neq i} x_{ji} = 1 && \forall i \in N, && (2) \\ &&& \sum_{j \in N, j \neq i} f_{ij}^k - \sum_{j \in N, j \neq i} f_{ji}^k = 0 && i \in N \setminus \{1\}, k \in \{2, \dots, n\} && (3) \\ &&& \sum_{i=2}^n f_{1i}^k = 1, && k \in \{2, \dots, n\}, && (4) \\ &&& \sum_{i=1, i \neq k}^n f_{ik}^k = 1, && k \in \{2, \dots, n\}, && (5) \\ &&& 0 \leq f_{ij}^k \leq x_{ij} && i \neq j \in N \times N, k \in \{2, \dots, n\}. && (6) \\ &&& x_{ij} \in \{0, 1\}, && i \neq j \in N \times N. \end{aligned}$$

El significado de las restricciones es el siguiente:

(1) – (2) Mantienen el significado ya dado en las anteriores formulaciones.

(3) Si el flujo llega a un nodo que no es su objetivo, entonces debe volver a salir.

- (4) Todas las unidades de flujo deben de salir del primer nodo.
- (5) Las unidades de flujo con destino k deben llegar a k .
- (6) El flujo solo se puede mover por las aristas del camino.

Esta formulación sigue siendo *compacta*, con un número de restricciones y de variables del orden de $O(n^3)$, y es bastante interesante por asegurar, a nivel teórico, que el *Lower Bound* de su problema relajado es el mismo que obtendríamos con la relajación de **TSP-DFJ** —cuyo *LB* sabemos es muy bueno—.

2.1.4. Formulación *Time-Staged* (TSP-TS)

Para terminar con las formulaciones del problema original podemos cambiar la filosofía y atacar el problema de la formación de *subtours* realizando un seguimiento de qué arista se activa en qué momento del circuito: si consideramos la salida de la primera ciudad como la etapa 1, buscamos saber qué arista debe recorrerse en la etapa k -ésima, para $k = 2, \dots, n$. Así, imponiendo salir en la etapa $k + 1$ del nodo al que entramos en la etapa k , tendremos asegurado que no se formarán ciclos cerrados.

De esta forma, llegamos a la siguiente formulación[‡]:

Variables:	Datos:
$r_{ij}^k = \begin{cases} 1, & \text{La arista } (i, j) \\ & \text{es la } k\text{-ésima del } \textit{tour}. \\ 0, & \text{en otro caso.} \end{cases}$	c_{ij} : el coste de mover flujo de i a j .

$$\begin{aligned}
 &\text{Minimizar} \quad \sum_{i=2}^n c_{1i} r_{1i}^1 + \sum_{k=2}^{n-1} \sum_{i,j=2, i \neq j}^n c_{ij} r_{ij}^k + \sum_{i=2}^n c_{i1} r_{i1}^n \\
 &\text{sujeto a} \quad \sum_{j=2}^n r_{1j}^1 = 1 \tag{1} \\
 &\quad \sum_{j=2}^n r_{j1}^n = 1 \tag{2} \\
 &\quad \sum_{k=1}^n \sum_{j \neq i}^n r_{ji}^k = 1, \quad 1 \leq i \leq n \tag{3} \\
 &\quad \sum_{j \neq i}^n r_{ji}^k - \sum_{j \neq i}^n r_{ij}^{k+1} = 0, \quad 2 \leq i \leq n, \quad 1 \leq k \leq n-1, \tag{4} \\
 &\quad r_{ij}^k \in \{0, 1\} \quad i \neq j \in N \times N, \quad k \in N
 \end{aligned}$$

Los significados de cada conjunto de restricciones son los siguientes:

[‡]Esta formulación ha sido consultada en [2], ya que el artículo que estábamos siguiendo contenía algún error.

- (1) El camino debe comenzar en el nodo 1.
- (2) El camino debe terminar en el nodo 1.
- (3) Debemos visitar todos los nodos una única vez.
- (4) Si entramos en un nodo en la etapa k , entonces en la etapa $k + 1$ salimos de él.

Ahora el número de restricciones disminuye respecto de la formulación **TSP-MCF**, siendo del orden de $O(n^2)$ únicamente. El comportamiento esperado sería que el *Lower Bound* del problema relajado se encuentre, en este caso, en un punto intermedio entre el que obtendríamos con la formulación **TSP-SCF** y la de **TSP-DFJ**.

A modo de resumen, incluimos en la Tabla 1 el orden del número de variables y de restricciones, así como la fuerza de la relajación lineal del problema para cada formulación. Por esto último entendemos que, si la relajación es fuerte para una determinada formulación, entonces se obtiene una cota inferior mejor (más grande) al resolver el problema relajado que para una formulación con relajación débil o media.

Formulación	Variables	Restricciones	Fuerza de la relajación al Problema Lineal
TSP-SCF	$O(n^2)$	$O(n^2)$	débil
TSP-TS	$O(n^3)$	$O(n^2)$	media
TSP-MCF	$O(n^3)$	$O(n^3)$	fuerte
TSP-DFJ	$O(n^2)$	$O(2^n)$	fuerte

Tabla 1: Comparativa del número de restricciones y de variables de cada una de las formulaciones presentadas del problema del viajante.

2.2. Problema de Steiner (STSP)

Veremos ahora una generalización del *problema del viajante* conocido como *Problema del viajante de Steiner* (o, *Steiner Travelling Salesman Problem*, **STSP**).

Consideremos un grafo simple y sin lazos, pero no necesariamente completo $\mathcal{G} = (N, G)$ (es decir, puede haber nodos que no estén conectados entre sí directamente por una arista), donde N es el conjunto de nodos que el viajante puede visitar, y G las aristas por las que se puede mover. Además, tenemos una serie de nodos prefijados $N_G \subseteq N$ que el viajante debe visitar obligatoriamente. Tenemos también costes c_{ij} asociados a cada arista $\{i, j\}$, considerando $c_{ij} = c_{ji}$. Así, el objetivo del problema es: *encontrar el camino de menor coste en el grafo, saliendo de un nodo inicial y volviendo a él, que pase por todos los nodos requeridos $i \in N_G$, permitiendo repetir aristas y pasar varias veces por el mismo nodo.*

Para la solución de este problema planteamos tres posibles formulaciones: la *formulación de Fleischmann* (**STSP-F**), que es el modelo clásico pero *no compacto*; la formula-

ción basada en flujos *Single-Commodity* (**STSP-SCF**) y la formulación *tipo Time-Staged* (**STSP-TS**). Las dos últimas formulaciones mencionadas tratan de mejorar a la primera atacando el problema desde filosofías distintas, pero en ambos casos buscando reducir el tamaño del problema.

2.2.1. Formulación de Fleischmann (STSP-F)

Este es el modelo clásico para el problema de Steiner, y puede verse como el análogo de la formulación de Dantzig-Fulkerson-Johnson del problema **TSP**. Para su formulación debemos introducir una notación nueva: dado un conjunto $S \subset N$, denotamos por $\delta(S)$ al conjunto de aristas que *salen* de S . Concretamente,

$$\delta(S) = \{g = \{i, j\} \in G : i \in S, j \notin S, \text{ o bien } j \in S, i \notin S\}.$$

Así, la formulación que obtenemos es la siguiente:

Variables:

Datos:

x_g : Número de veces que se usa la arista $g \in G$.
 c_g : coste de emplear la arista $g \in G$.

$$\begin{aligned} &\text{Minimizar} && \sum_{g \in G} c_g x_g \\ &\text{sujeto a} && \sum_{g \in \delta(S)} x_g \geq 2 && \text{Para cada } S \subset N \text{ t.q. } S \cap N_R \neq \emptyset, N_R \setminus S \neq \emptyset, \quad (1) \\ &&& \sum_{g \in \delta(\{i\})} x_g \equiv 0 \pmod{2}, \quad i \in N, \quad (2) \\ &&& x_g \in \mathbb{Z}_+ && g \in G. \end{aligned}$$

La motivación de cada restricción se da a continuación:

- (1) *Restricción de eliminación de subtours*: si un subconjunto de nodos contiene algún nodo requerido, y fuera del conjunto hay otro nodo requerido, entonces al menos dos aristas deben cruzar dicho subconjunto.
- (2) Asegura que, al entrar en un nodo, salgamos de él, causando que el grado de cada nodo sea par[§].

Al igual que ocurre con la formulación **TSP-DFJ**, esta es una formulación con unos órdenes de magnitud muy elevados, sobre todo en cuanto al número de restricciones, que va a crecer exponencialmente con el número de nodos requeridos: tenemos $|G|$ variables y un número de restricciones del orden de $O(2^{|N_G|})$. Es evidente entonces que es un modelo *no compacto*, y por tanto no muy eficiente a nivel de memoria ni de tiempo de resolución, pero que tendrá como gran ventaja —a nivel teórico— la fuerza de su relajación a un

[§]El *grado* de un nodo es el número de aristas que inciden en él.

problema lineal, cerrando el *Lower Bound* y el *Upper Bound* con menos iteraciones de *branch and bound*.

2.2.2. Formulación tipo flujo *Single-Commodity* (STSP-SCF)

Denotemos D_G al conjunto de aristas orientadas del grafo $\mathcal{G} = (N, G)$ (esto es, los pares de aristas $(i, j), (j, i)$ tal que $\{i, j\} \in G$). Las otras dos formulaciones que veremos se basan en el siguiente principio:

Se puede probar que, para cualquier solución óptima del problema de Steiner, ninguna arista se recorre más de una vez en cada dirección.

Esto significa que cada arista orientada $(i, j) \in D_G$ se recorre, a lo sumo, una vez (notemos que consideramos cada dirección de la arista por separado, pudiendo recorrer la arista en el sentido (i, j) y después volver en sentido (j, i)).

Este resultado es de gran importancia para las formulaciones que veremos, pues permite dejar de considerar las variables enteras x_g de la formulación **STSP-F** y pasar a considerar variables binarias que indiquen si se recorre o no un determinado sentido de cada arista, como veremos a continuación.

Para la formulación **STSP-SCF** se plantea una idea muy similar a las ideas de flujos para el problema original del viajante: partiremos del nodo inicial —el cual consideraremos el nodo 1, que será uno de los requeridos— con $|N_G| - 1$ unidades de flujo. Buscaremos ir dejando una unidad de flujo en cada uno de los nodos requeridos, pasando por las aristas orientadas que sean necesarias, que por el punto antes explicado, sabemos que no serán usadas más de una vez.

Así, la formulación que nos quedaría sería la siguiente:

Variables:	Datos:
$\tilde{x}_a = \begin{cases} 1, & \text{si se recorre la arista} \\ & \text{orientada } a \in D_G, \\ 0, & \text{en otro caso.} \end{cases}$	c_g : coste de emplear la arista $a \in D_G$.

f_a : flujo que circula por la arista $a \in D_G$.

$$\begin{aligned}
& \text{Minimizar} && \sum_{a \in D_G} c_a \tilde{x}_a \\
& \text{sujeto a} && \sum_{a \in \delta^+(i)} \tilde{x}_a \geq 1, && i \in N_G, && (1) \\
& && \sum_{a \in \delta^+(i)} \tilde{x}_a = \sum_{a \in \delta^-(i)} \tilde{x}_a, && i \in N, && (2) \\
& && \sum_{a \in \delta^-(i)} f_a - \sum_{a \in \delta^+(i)} f_a = 1, && i \in N_G \setminus \{1\}, && (3) \\
& && \sum_{a \in \delta^-(i)} f_a - \sum_{a \in \delta^+(i)} f_a = 0, && i \in N \setminus N_G, && (4) \\
& && 0 \leq f_a \leq (|N_G| - 1) \tilde{x}_a, && a \in D_G, && (5) \\
& && \tilde{x}_a \in \{0, 1\}, && a \in D_G.
\end{aligned}$$

donde

- $\delta^+(i)$ denota todas las aristas que llegan al nodo i ,
- $\delta^-(i)$ son todas las aristas que salen del nodo i .

Las explicaciones de cada una de las restricciones son las siguientes:

- (1) Si el nodo i es requerido, entonces el *tour* debe salir al menos una vez de i .
- (2) El número de aristas por las que pasa el viajante que entran a un nodo debe ser igual al número de aristas que salen (salimos las mismas veces que entramos).
- (3) En cada nodo requerido dejamos una única unidad de flujo, salvo en el primer nodo.
- (4) Cuando un nodo no es requerido, no se deja ninguna unidad de flujo en él.
- (5) Si una arista no se usa, el flujo que pasa por dicha arista es nulo; en otro caso, la arista aparece en la solución.

Ahora la formulación del problema sí es *compacta*, con un número de variables y de restricciones del orden de $O(|G|)$. Este problema será mucho más fácil de tratar que la formulación no compacta **STSP-F**. El precio a pagar, teóricamente, sería una reducción en la fuerza de la relajación lineal, cayendo su *Lower Bound* por debajo del de la formulación de Fleischmann.

2.2.3. Formulación *Time-Staged* (STSP-TS)

Empleando los mismos principios que vimos con anterioridad, podemos extender la idea que ya vimos en la formulación **TSP-TS**. De esta forma, sabiendo que el número de pasos necesarios para completar el camino está acotado por el número de aristas que podemos

recorrer,

$$\sum_{g \in G} x_g \leq |D_G| = 2|G|,$$

podemos definir variables binarias que nos indiquen si se usa o no una arista $a \in D_G$ en la etapa k , siendo el máximo de etapas $k = |D_G|$. Así, la formulación quedaría como sigue:

Variables:

Datos:

$$r_a^k = \begin{cases} 1, & \text{si se recorre la arista } a \\ & \text{en la etapa } k \\ 0, & \text{en otro caso.} \end{cases} \quad c_g: \text{coste de emplear la arista } a \in D_G.$$

$$\begin{aligned} \text{Minimizar} \quad & \sum_{k=1}^{|D_G|} \sum_{a \in D_G} c_a r_a^k \\ \text{sujeto a} \quad & \sum_{a \in \delta^+(1)} r_a^1 = 1, \end{aligned} \tag{1}$$

$$r_a^1 = 0, \quad a \in D_G \setminus \delta^+(1), \tag{2}$$

$$\sum_{k=1}^{|D_G|} \sum_{a \in \delta^+(1)} r_a^k = \sum_{k=1}^{|D_G|} \sum_{a \in \delta^-(1)} r_a^k, \tag{3}$$

$$\sum_{k=1}^{|D_G|} \sum_{a \in \delta^+(i)} r_a^k \geq 1, \quad i \in N_G, \tag{4}$$

$$\begin{aligned} \sum_{a \in \delta^-(i)} r_a^k &\geq \sum_{a \in \delta^+(i)} r_a^{k+1}, \quad i \in N, \quad k = 1, \dots, |D_G| - 1, \\ r_a^k &\in \{0, 1\}, \quad a \in D_G, \quad k = 1, \dots, |D_G|. \end{aligned} \tag{5}$$

- (1) Debemos salir del nodo origen 1 en el primer paso, por exactamente una arista.
- (2) Solo podemos salir del nodo 1 en el primer paso.
- (3) Para que el recorrido empiece y termine en el nodo 1, por cada vez que salimos debemos volver a entrar.
- (4) Visitaremos por lo menos una vez cada nodo requerido (de cada nodo requerido $i \in N_G$, salimos por lo menos una vez).
- (5) Para poder salir de un nodo i en el paso $k + 1$, debemos haber llegado a él en la etapa k . Esto impide salir de nodos a los que no hemos llegado, asegurando que toda solución factible sea un camino conexo. [†]

Es importante notar que, a nivel de memoria, sería interesante encontrar mejores cotas

[†]Esta restricción está escrita con una igualdad en [1]. Realizando los estudios computacionales, nos dimos cuenta de que necesitábamos una desigualdad, ya que si tuviéramos la igualdad, forzamos a que exista alguna arista a tal que $r_a^k > 0$ para todo k , y entonces el camino tendría siempre longitud $|D_G|$, pero esta es una cota superior, lo que da lugar a soluciones no óptimas.

para para el número de etapas totales que necesita realizar el proceso. Con lo descrito hasta ahora, tendríamos una cantidad máxima igual al número de aristas del grafo orientado — lo cual es excesivo—. Sin embargo, se puede probar ([1]) que realmente solo son necesarios, como mucho $2(|N| - 1)$ pasos para completar el camino (es decir, basta con pasar como máximo pasar dos veces por cada nodo distinto del nodo de partida). De esta forma, conseguimos reducir bastante el número de restricciones.

Aún así, con la modificación anterior, tenemos un número de restricciones y variables algo mayor al que teníamos para la anterior formulación (**STSP-SCF**), con órdenes de $O(|N||G|)$ para las variables y $O(|N|^2)$ para las restricciones. Mantenemos la propiedad de ser *compacta*, eso sí. En [1] se conjetura que la cota inferior de la relajación al problema lineal es, para esta formulación, intermedia respecto a las otras dos formulaciones vistas.

A modo de resumen, tenemos en la Tabla 2 un resumen del número de variables y restricciones de cada formulación, así como la fuerza teórica de la relajación al problema lineal. Entendemos por esto último lo mismo que explicamos en la subsección anterior (ver Tabla 1).

Formulación	Variables	Restricciones	Fuerza de la relajación al Problema Lineal
STSP-F	$ G $	$O(2^{ N_G })$	Fuerte
STSP-SCF	$O(G)$	$O(G)$	débil
STSP-TS	$O(N G)$	$O(N ^2)$	intermedia (conjetura)

Tabla 2: Comparativa del número de restricciones y de variables de cada una de las formulaciones que presentamos para la resolución del problema del viajante de Steiner.

3. Estudio computacional

En esta sección nos centraremos en realizar un estudio computacional para comprobar empíricamente los resultados obtenidos en la sección anterior en cuanto al rendimiento de las distintas formulaciones. Para ello, nos centraremos en cada problema por separado y generaremos instancias de distintos tamaños utilizando , que emplearemos para poner a prueba cada una de las formulaciones implementadas en . Comprobaremos aspectos como el tiempo de convergencia del solver **Gurobi**, o en caso de no llegar a converger en un máximo de 5 minutos, el *lower bound* al que llega cada una de las formulaciones, así como el *gap* relativo (al que nos referiremos simplemente como *gap*) y la mejor solución encontrada, comparándolas gráficamente.

Además, resolveremos los problemas relajados en cada una de las instancias para comprobar de manera práctica la fuerza que tienen, entendiendo por esto cómo de buena es la cota inferior a la que se llega.

3.1. Problema del viajante

Comenzaremos estudiando las cuatro formulaciones del *problema del viajante* que hemos presentado en la sección anterior. Para ello, utilizaremos el generador de instancias que se encuentra en el archivo adjunto **Generador_TSP.R**. En la Tabla 3 encontramos el resumen de las ejecuciones para diferentes valores de $n = |N|$ del *problema del viajante*, empleando en todos los casos el solver **Gurobi** con un tiempo límite de 300s. Además, en la Tabla 4 mostramos los resultados de resolver el problema lineal relajado de cada una de las formulaciones, con el objetivo de ver su comportamiento en términos del *Lower Bound*. Junto a cada formulación indicamos también su *nivel de fuerza* teórico.

$ N $	Formulación	n° Variables	n° Restricciones	<i>Lower Bound</i>	<i>Gap</i>	Tiempo
10	TSP-SCF	180	120	152.08	0.0000	0.047 seg.
	TSP-TS	900	93	152.08	0.0000	0.469 seg.
	TSP-MCF	990	920	152.08	0.0000	0.281 seg.
	TSP-DFJ	90	1032	152.08	0.0000	0.328 seg.
20	TSP-SCF	760	440	163.76	0.0000	0.078 seg.
	TSP-TS	7 600	383	163.76	0.0000	0.328 seg.
	TSP-MCF	7 980	7 640	163.76	0.0000	0.750 seg.
	TSP-DFJ	380	1 048 594	163.76	0.0000	83.031 seg.
50	TSP-SCF	7 080	3 720	150.09	0.0000	1.109 seg.
	TSP-TS	212 400	3 543	150.09	0.0000	161.016 seg.
	TSP-MCF	215 940	212 520	150.09	0.0000	61.000 seg.
80	TSP-SCF	12 640	6 560	177.68	0.0000	2.375 seg.
	TSP-TS	505 600	6 323	176.23	0.1575	304.156 seg.
	TSP-MCF	511 920	505 760	177.69	0.0000	246.640 seg.
100	TSP-SCF	19 800	10 200	176.61	0.0000	3.703 seg.
	TSP-TS	990 000	9 903	175.87	0.0685	308.516 seg.
	TSP-MCF	999 900	990 200	176.06	0.9638	308.860 seg.

Tabla 3: Resultados de la resolución del problema **TSP** para distintas instancias y formulaciones.

Podemos observar que, en la primera instancia, con un total de nodos de $n = 10$, **Gurobi** termina en menos de un segundo con todas las formulaciones. Presentamos en la Figura 2 la solución obtenida con cada una de las formulaciones. Siendo esta la misma para todas ellas. Además, si nos fijamos en los resultados de los problemas relajados en este caso, podemos ver que todos realmente son capaces de alcanzar el óptimo en el primer nodo, muy posiblemente porque el problema es tan sencillo que no ofrece dificultad ninguna.

Por otro lado, con $n = 20$ nodos, comenzamos a observar el mal rendimiento de la formulación **TSP-DFJ** frente a las otras tres, notando un tiempo de resolución de 83 segundos por parte del *solver*, mientras que para el resto de formulaciones se resuelve el problema en menos de un segundo. Esto es debido al escalado exponencial del número de restric-

$ N $	Formulación	<i>Lower Bound</i> de la relajación	Fuerza teórica
10	TSP-SCF	152.08	débil
	TSP-TS	152.08	intermedia
	TSP-MCF	152.08	fuerte
	TSP-DFJ	152.08	fuerte
20	TSP-SCF	161.61	débil
	TSP-TS	163.76	intermedia
	TSP-MCF	163.76	fuerte
	TSP-DFJ	163.76	fuerte
50	TSP-SCF	147.56	débil
	TSP-TS	147.65	intermedia
	TSP-MCF	148.00	fuerte
80	TSP-SCF	175.71	débil
	TSP-TS	175.88	intermedia
	TSP-MCF	177.00	fuerte
100	TSP-SCF	175.69	débil
	TSP-TS	175.77	intermedia
	TSP-MCF	176.06	fuerte

Tabla 4: Solución para las cinco instancias ejecutadas del *problema del viajante* de los respectivos problemas relajados de cada formulación.

ciones. En este caso vemos cómo pasa de alrededor de $2^{10} = 1024$ en la primera instancia a unas $2^{20} = 1048576$, tan solo aumentando 10 nodos. Cabe destacar que a partir de este punto resulta imposible resolver los problemas con  utilizando la formulación **TSP-DFJ**, pues los conjuntos potencia son demasiado grandes como para almacenar todas las restricciones en memoria.

En cuanto a los problemas relajados de la segunda instancia, podemos apreciar que la formulación de flujo *Single-Commodity* es la única que no alcanza el óptimo, lo cual contrasta en un primer momento con los resultados de la resolución en la Tabla 3, pues es la formulación que primero termina. Esto nos indica dos cosas: en efecto, la formulación **TSP-SCF** es la que tiene un *Lower Bound* más débil al resolver la relajación, es decir, usualmente será menor a las del resto (aunque lo contrastaremos viendo los resultados de las próximas instancias), pero lo compensa gracias a su reducido tamaño en comparación a cualquiera de las otras formulaciones, lo que le permite terminar primero.

Comentaremos a continuación lo que ocurre en los casos $n = 50, 80$ y después realizaremos comentarios sobre los problemas relajados.

Aumentando más el número de nodos, notamos más claramente la gran diferencia entre formulaciones, siendo la formulación **TSP-SCF** la clara ganadora. Si tenemos $n = 50$ nodos, el solver converge en tan solo 1 segundo con dicha formulación, mientras que tarda un minuto con la formulación **TSP-MCF**, y más de dos minutos y medio con la

formulación *Time-Staged*, a pesar de que **TSP-MCF** presenta muchas más restricciones que **TSP-TS**.

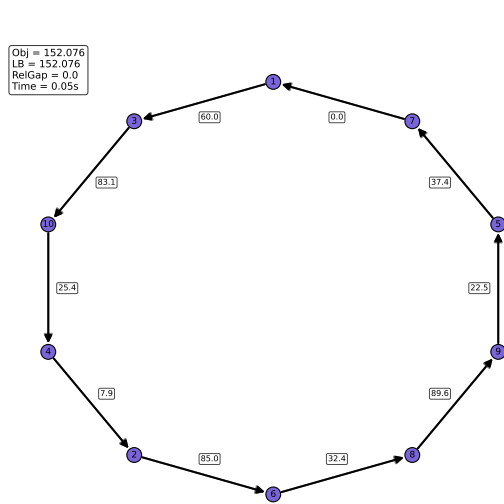
Resultados similares a los de la anterior instancia se obtienen al aumentar a $n = 80$ nodos, donde con **TSP-SCF** Gurobi tarda solo 2 segundos en encontrar la solución óptima, pero *Time-Staged* ni siquiera converge en cinco minutos, resultando en este caso peor en rendimiento que la formulación *Multicommodity flow*. En la Figura 3 se observan gráficamente las soluciones que terminó encontrando Gurobi en 5 minutos; para las instancias con $n = 50, 80$ nodos. Notamos que la solución de **TSP-TS** es, en efecto, distinta a la encontrada con las otras dos formulaciones.

Pasando a comentar los problemas relajados en los dos últimos casos comentados, apreciamos un comportamiento muy cercano al teórico: **TSP-SCF** presenta el menor *Lower Bound*, mientras que la formulación de *Multicommodity flow* presenta la mayor cota inferior, estando la formulación *Time-Staged* entre medias. Aún así, las diferencias son mínimas en este caso. También, es interesante revisar los tiempos de resolución de las formulaciones y contrastarlas con el *Lower Bound* del que parten en la primera iteración de *branch and bound*, sobre todo en el caso de la formulación *Time-Staged*: como vemos en las filas tercera y cuarta de la Tabla 4, el punto de partida es bastante cercano al valor de la función objetivo en el óptimo, por lo que es posible que se esté dando una de dos situaciones: o le está resultando muy complicado elevar el *Lower Bound*, o le está costando reducir el *Upper Bound*. Al revisar pormenorizadamente las distintas iteraciones —permitiendo a Gurobi mostrar en consola toda la información— detectamos que en un inicio el problema es doble, no puede mejorar ninguno, pero llegados a un punto es la cota inferior la que es incapaz de mejorar, mientras que la superior ya se encuentra en el punto óptimo. Tal vez esta formulación se beneficiaría de una búsqueda *a lo ancho* —*breadth-first search*— o incluso de un método mixto, en el que llegados a la mitad de la ejecución se intercambie el algoritmo de búsqueda y pasemos a aumentar el *Lower Bound* al resolver nodos en el mismo nivel.

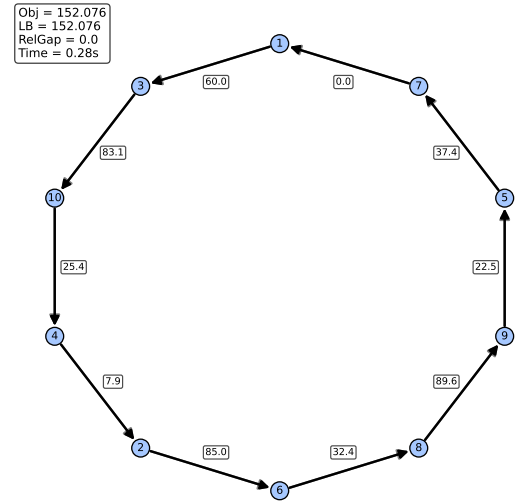
Sin embargo, aunque parezca que la formulación **TSP-MCF** tiene un rendimiento intermedio entre **TSP-SCF** y **TSP-TS**, el rápido escalado del número de restricciones llega a saturar al *solver* una vez hay una gran cantidad de nodos. En efecto, en el caso $n = 100$, observamos en la Tabla 3 que *Multicommodity Flow* presenta un *gap* relativo muy grande tras agotar los 5 minutos de tiempo límite, a pesar de acercarse al óptimo en términos de *Lower Bound*, mientras que **TSP-TS** sí consigue encontrar una solución factible razonable. De nuevo, la formulación **TSP-SCF** es la que mejor rendimiento muestra, llegando a la solución óptima en tan solo 3.7 segundos.

Una última observación que podemos hacer sobre la instancia $n = 100$ al revisar los problemas relajados, es precisamente sobre el mal comportamiento de la formulación *Multicommodity-flow*. Vemos que esta formulación no es capaz de llegar a cerrar el *gap* en el tiempo prefijado, y notamos que ya en el primer nodo el *Lower Bound* al que llega es con el que se va a mantener el resto de la resolución. Al revisar la salida de Gurobi, hemos notado que el problema ni siquiera pasa de primer nodo, gastando gran parte de su

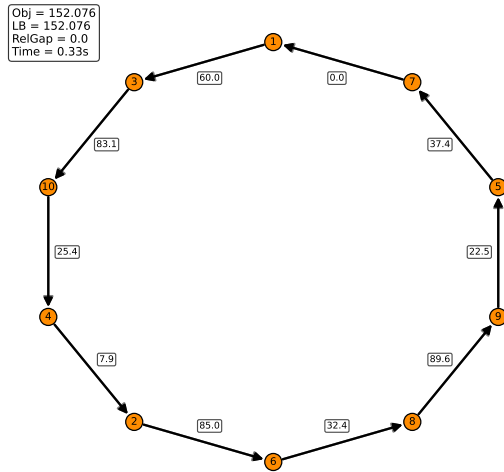
tiempo con diversos algoritmos de preparación para reducir el tamaño del problema. No sucede lo mismo con **TSP-TS**, que en este caso sí llega a entrar en el método de *Branch and Bound* tras resolver relativamente rápido los diversos algoritmos de preparación del *solver*. Esto nos indica que, muy probablemente, es el número de restricciones lo que condiciona negativamente el problema, al tener ambas formulaciones un número similar de variables, pero *Time-Staged* muchas menos restricciones. Aún así, no es capaz de mejorar, en el tiempo restante, ni el *Lower Bound* ni el *Upper Bound*.



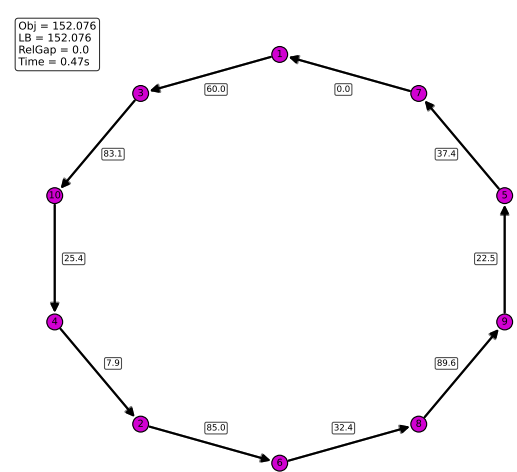
(a) Solución óptima encontrada por Gurobi en el problema **TSP** con la formulación *Single-Commodity Flow*.



(b) Solución óptima encontrada por Gurobi en el problema **TSP** con la formulación *Multicommodity Flow*.



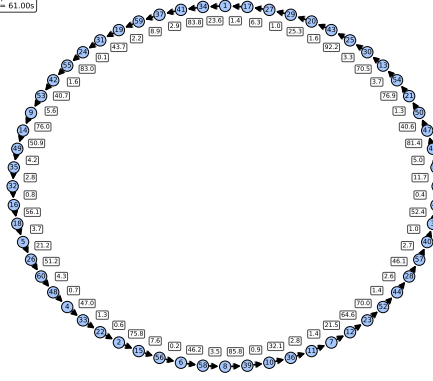
(c) Solución óptima encontrada por Gurobi en el problema **TSP** con la formulación de *Dantzig-Fulkerson-Johnson*.



(d) Solución óptima encontrada por Gurobi en el problema **TSP** con la formulación *Time-Staged*.

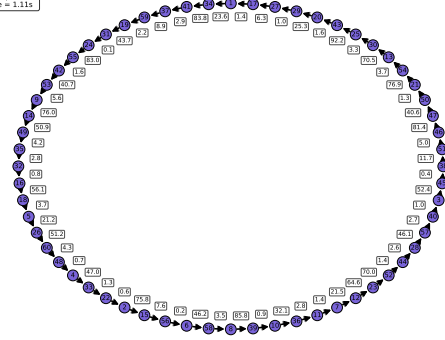
Figura 2: Soluciones de la instancia 1 del problema **TSP** para las cuatro formulaciones estudiadas. Se representa el coste de cada arista encima de la misma.

Obj = 150.088
LB = 150.088
RelGap = 0.0
Time = 61.00s



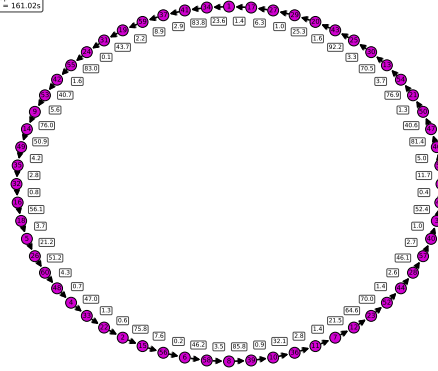
(a) *Multicommodity Flow*, $n = 50$.

Obj = 150.088
LB = 150.088
RelGap = 0.0
Time = 1.11s



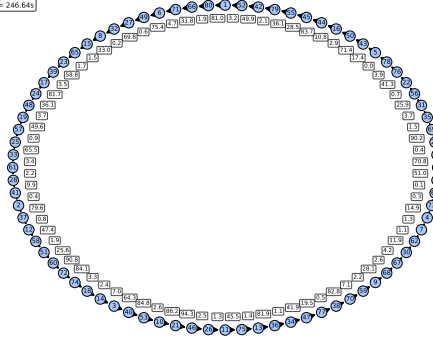
(b) *Single-commodity Flow*, $n = 50$.

Obj = 150.088
LB = 150.088
RelGap = 0.0
Time = 161.02s



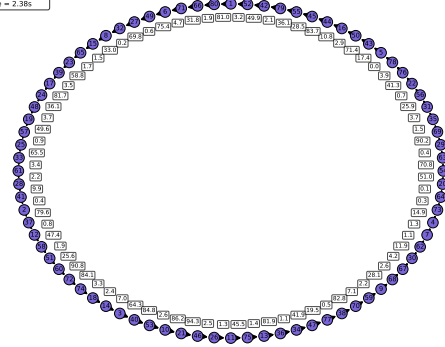
(c) *Time-Staged*, $n = 50$.

Obj = 177.685
LB = 177.685
RelGap = 0.0
Time = 246.64s



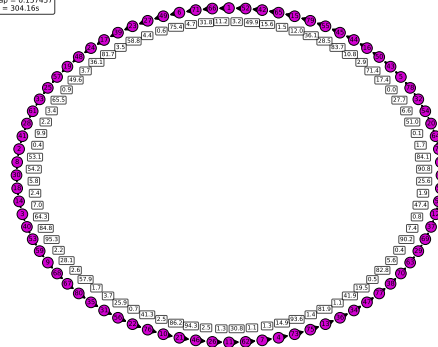
(d) *Multicommodity Flow*, $n = 80$.

Obj = 177.685
LB = 177.678923
RelGap = 3.4e-05
Time = 2.38s



(e) *Single-commodity Flow*, $n = 80$.

Obj = 209.17
LB = 176.234784
RelGap = 0.127457
Time = 304.16s



(f) *Time-Staged*, $n = 80$.

Figura 3: Soluciones para las instancias $n = 50, 80$ del problema TSP.

3.2. Problema de Steiner

Al igual que con el problema del viajante, comenzamos con una instancia pequeña, imponiendo en nuestro generador (cuyo comportamiento se detalla en el propio archivo `Generador_STSP.R`) un total de 18 nodos, con una probabilidad de generar arista de 0.2, y una probabilidad de nodo requerido de 0.4. Los resultados de la ejecución se encuentran resumidos en la Tabla 5, y se pueden ver gráficamente las soluciones encontradas por el solver `Gurobi` con cada formulación en la Figura 4.

Formulación	n° Variables	n° Restricciones	<i>Lower Bound</i>	<i>Gap</i>	Tiempo
STSP-SCF	164	126	101.00	0.0000	0.078 seg.
STSP-TS	2 788	684	101.00	0.0000	2.125 seg.
STSP-F	59	261 138	101.00	0.0000	5.515 seg.

Tabla 5: Resultados de la primera instancia del problema de Steiner.

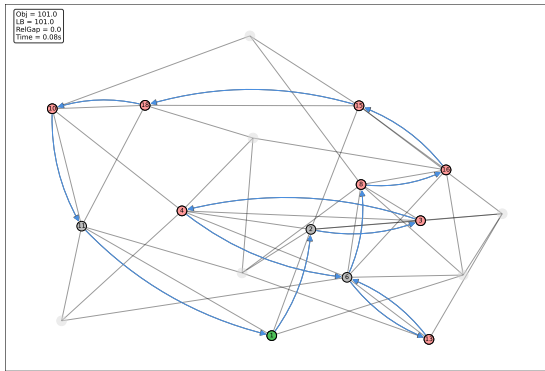
Formulación	<i>Lower Bound</i> de la relajación	Fuerza teórica
STSP-SCF	87.50	débil
STSP-TS	85.45	intermedia (conjetura)
STSP-F	100.50	fuerte

Tabla 6: Soluciones del problema relajado relativo a la primera instancia del problema de Steiner.

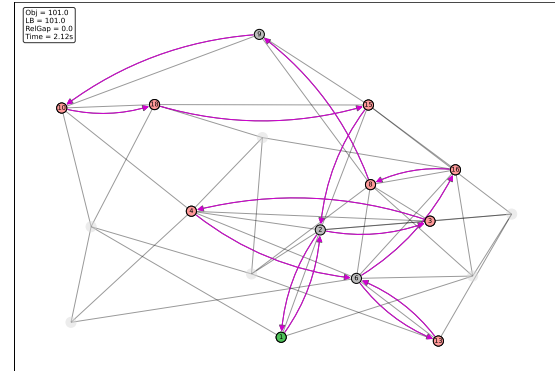
Observamos que en las tres formulaciones se encuentran soluciones equivalentes, con valor de la función objetivo 101.00 en cada caso. Sin embargo, se observa que, con la formulación **STSP-SCF**, `Gurobi` termina en 0.078 segundos, mientras que con la formulación **STSP-TS** tarda 2 segundos más. Por otro lado, la formulación de Fleischmann parece mostrar un peor rendimiento, con un *elapsed time* de más de 5 segundos. Esto es causa del elevado número de restricciones, habiendo en este caso un total de 261 138.

Ahora bien, en la Tabla 6 podemos ver las soluciones de los problemas relajados asociados a cada una de las formulaciones, y se aprecia mejor la utilidad de formulaciones más estrictas frente a otras que pueden ser menos restrictivas. Como vemos, la resolución del problema relajado asociado a **STSP-F** está muy cerca de la solución real, mientras que las otras dos formulaciones presentan *Lower Bounds* más alejadas. Curiosamente, apreciamos cómo la solución relajada de *Single-Commodity flow* es mejor que la de *Time-Staged*, lo cual no es congruente con los resultados teóricos conjeturados.

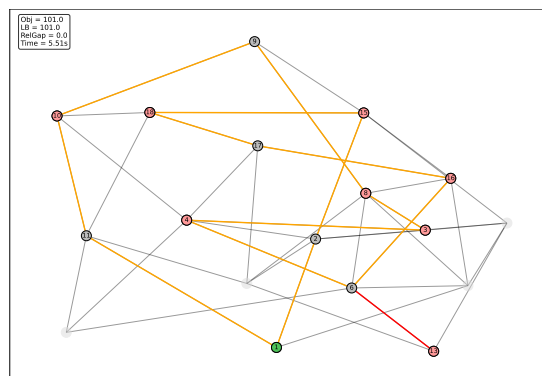
Para la segunda instancia, aumentaremos ligeramente el número de nodos, hasta los 21, con una probabilidad de arista de 0.2, y una probabilidad de nodo requerido de 0.2. Los resultados de `Gurobi` se encuentran en la Tabla 5, y la representación gráfica de la solución en la Figura 5. Podemos notar cómo añadiendo unos pocos nodos, el tiempo que tarda `Gurobi` en resolver la formulación de Fleischmann aumenta enormemente respecto a la instancia anterior, siendo ahora de 52 segundos. Esto es debido a que las restricciones



(a) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación *Single-Commodity Flow*.



(b) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación *Time-Staged*.



(c) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación de Fleischmann. Las líneas naranjas indican que la arista se recorre una única vez, mientras que las líneas rojas indican que la arista se recorre dos veces.

Figura 4: Soluciones de la instancia 1 del problema **STSP** para las tres formulaciones estudiadas. En rojo se representan los nodos requeridos, en verde el nodo inicial, y en gris los nodos no requeridos.

aumentan exponencialmente, siendo ahora más de dos millones, multiplicándose por 8 el número de restricciones de la instancia anterior, pues hemos aumentado 3 nodos, y $2^{21}/2^{18} = 2^3 = 8$. Por otro lado, aunque aumenta también el número de variables de la formulación *Time-Staged*, no parece tardar más en encontrar una solución óptima. La formulación basada en flujo *Single-Commodity* tampoco empeora, y tan solo aumenta en 40 variables y 23 restricciones respecto de la instancia anterior. En este caso, las tres formulaciones han encontrado la misma solución, como se puede ver en la Figura 5.

Formulación	n° Variables	n° Restricciones	<i>Lower Bound</i>	<i>Gap</i>	Tiempo
STSP-SCF	204	149	61.00	0.0000	0.156 seg.
STSP-TS	4 080	925	61.00	0.0000	2.156 seg.
STSP-F	72	2 031 637	61.00	0.0000	52.204 seg.

Tabla 7: Resultados de la segunda instancia del problema de Steiner.

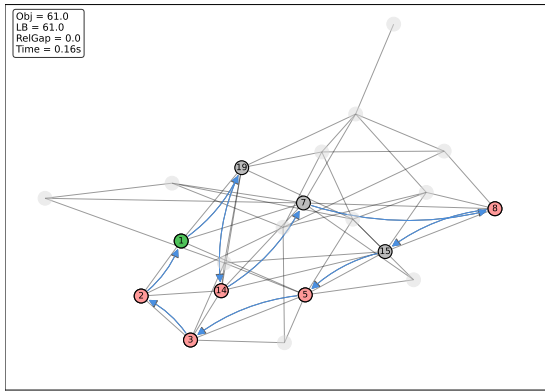
Formulación	<i>Lower Bound</i> de la relajación	Fuerza teórica
STSP-SCF	51.60	débil
STSP-TS	45.50	intermedia (conjetura)
STSP-F	61.00	fuerte

Tabla 8: Soluciones del problema relajado relativo a la segunda instancia del problema de Steiner.

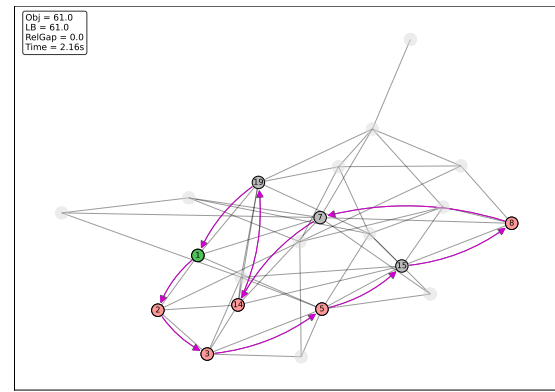
En la Tabla 8 presentamos de nuevo la resolución de los problemas relajados para esta instancia, y volvemos a ver la enorme ventaja de una formulación más fuerte. La relajación de la formulación de Fleischmann en esta ocasión ya llega a la misma solución óptima que resolviendo el problema lineal entero, es decir, realmente esta formulación solo necesita pasar en este caso por un nodo de *Branch and Bound* para encontrar el óptimo. El inconveniente es, obviamente, que la resolución de este único problema es computacionalmente muy costosa, lo que hace de esta ventaja poco útil. Por último, notamos cómo **STSP-SCF** sigue obteniendo una cota mejor que la de **STSP-TS**.

Para la siguiente instancia, vamos a poner a prueba las dos formulaciones que todavía no han dado problemas, *Time-Staged* y *Single-Commodity Flow*. Utilizaremos un grafo generado de 50 nodos, con una probabilidad de arista de 0.01, y una probabilidad de nodo requerido de 0.05. En la Tabla 9 se muestra el resumen de la ejecución, y en la Figura 6 la solución encontrada por Gurobi con las formulaciones **STSP-SCF** y **STSP-TS**. En este caso, no ha sido posible siquiera ejecutar la formulación de Fleischmann porque el tamaño del conjunto potencia del conjunto de nodos N excede el permitido por .

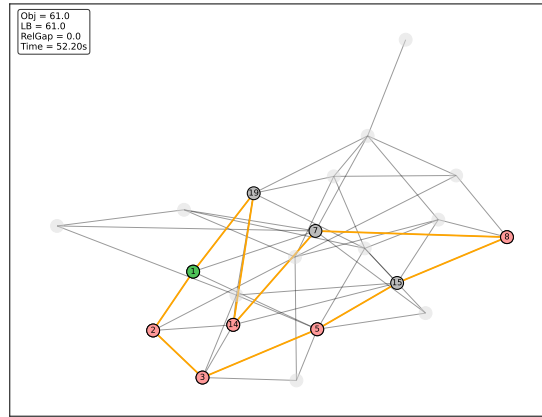
Observamos que, al aumentar bastante el número de nodos, la cantidad de variables y restricciones de la formulación *Time-Staged* crece considerablemente. En este caso, con **STSP-TS** se agota el tiempo de ejecución de Gurobi, de 5 minutos, pero a pesar de acabar con un *Gap* relativo de 0.2847, y una cota inferior de 103.00 para el valor de la función objetivo en el óptimo, realmente sí alcanza la solución óptima, que es la de la Figura 6b. Esto lo sabemos porque observamos la misma solución que **STSP-SCF** pero recorrida al



(a) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación *Single-Commodity Flow*.



(b) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación *Time-Staged*.

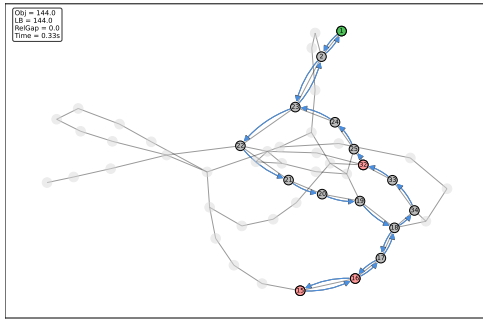


(c) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación de Fleischmann. Las líneas naranjas indican que la arista se recorre una única vez, mientras que las líneas rojas indican que la arista se recorre dos veces.

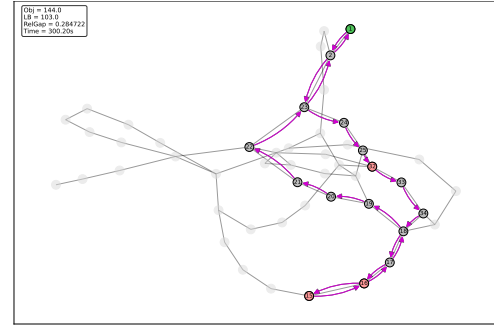
Figura 5: Soluciones de la instancia 2 del problema **STSP** para las tres formulaciones estudiadas. En rojo se representan los nodos requeridos, en verde el nodo inicial, y en gris los nodos no requeridos.

Formulación	n° Variables	n° Restricciones	<i>Lower Bound</i>	<i>Gap</i>	Tiempo
STSP-SCF	248	227	144.00	0.0000	0.328 seg.
STSP-TS	12 152	4 979	103.00	0.2847	300.203 seg.

Tabla 9: Resultados de la tercera instancia del problema de Steiner con 300s de tiempo límite para Gurobi.



(a) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación *Single-Commodity Flow*.



(b) Mejor solución encontrada por Gurobi en el problema **STSP** con la formulación *Time-Staged*.

Figura 6: Soluciones de la instancia 3 del problema **STSP** para las tres formulaciones estudiadas. En rojo se representan los nodos requeridos, en verde el nodo inicial, y en gris los nodos no requeridos.

Formulación	<i>Lower Bound</i> de la relajación	Fuerza teórica
STSP-SCF	119.33	débil
STSP-TS	31.81	intermedia (conjetura)

Tabla 10: Soluciones del problema relajado relativo a la tercera instancia del problema de Steiner.

revés, con el mismo valor de la función objetivo. Sin embargo, para *Time-Staged* notamos un rendimiento considerablemente peor que el de la formulación **STSP-SCF**, que sigue manteniendo el número de variables y restricciones bastante similar al de la instancia anterior, y se resuelve en menos de 1 segundo.

En términos de los resultados de los problemas relajados, presentados en la Tabla 10, no hay gran sorpresa. En la línea de lo que podíamos intuir por los resultados obtenidos en las anteriores instancias, la formulación *Single-Commodity flow* supera en *Lower Bound* a la de *Time-Staged*. Además, viendo lo cerca que está el *Lower Bound* de **STSP-SCF** respecto del óptimo, y considerando que Fleischmann tendría un resultado semejante a las anteriores instancias si contásemos con memoria suficiente —es decir, se encontraría muy cerca del óptimo—, podemos convencernos que el *Lower Bound* de ambas formulaciones son semejantes.

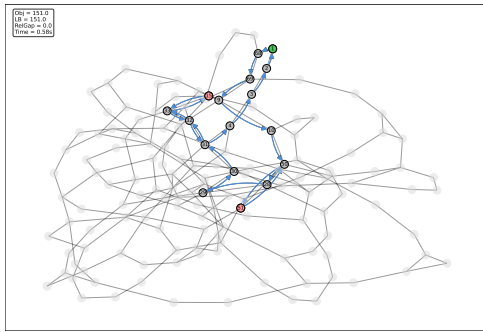
Como última nota, se puede llegar a apreciar un caso de *falta de progreso en la mejoría del Lower Bound* en la resolución del problema lineal entero de la formulación *Time-Staged*: empezando en un punto no muy alejado de la solución final en términos del valor de la función objetivo, pues no tarda mucho en alcanzar el óptimo en su *Upper Bound*, pero es incapaz de cerrar el *gap*, sufriendo de un caso de *flat Lower Bound* —tal vez sería recomendable cambiar los ajustes para que se priorice una búsqueda en los nodos *a lo ancho*, de forma que se priorice la mejoría del *Lower Bound* sobre encontrar el óptimo—.

Para la última instancia, vamos a generar un problema muy grande, con un total de

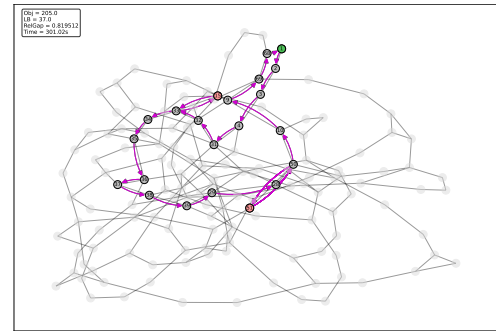
150 nodos, una probabilidad de arista de 0.05, y una probabilidad de nodos requeridos de 0.05 —notemos que habiendo 150 nodos, tendremos una gran cantidad de aristas—. Observamos la salida de  en la Tabla 11, y las soluciones representadas gráficamente en la Figura 7.

Formulación	n° Variables	n° Restricciones	<i>Lower Bound</i>	<i>Gap</i>	Tiempo
STSP-SCF	824	714	151.00	0.0000	0.578 seg.
STSP-TS	122 776	44 965	37.00	0.8195	301.016 seg.

Tabla 11: Resultados de la cuarta instancia del problema de Steiner relajado con 300s de tiempo límite para Gurobi.



(a) Solución óptima encontrada por Gurobi en el problema **STSP** con la formulación *Single-Commodity Flow*.



(b) Mejor solución encontrada por Gurobi en el problema **STSP** con la formulación *Time-Staged*.

Figura 7: Soluciones de la instancia 4 del problema **STSP** para las tres formulaciones estudiadas. En rojo se representan los nodos requeridos, en verde el nodo inicial, y en gris los nodos no requeridos.

Con estos resultados, confirmamos la gran diferencia de rendimiento entre la formulación **STSP-TS** y **STSP-SCF**. La primera, se detiene por tiempo límite, a los 5 minutos, obteniendo una solución factible con un valor de la función objetivo de 205.00, 54 unidades por encima de la solución óptima, encontrada por la formulación basada en flujo *Single-Commodity* en menos de un segundo.

Los resultados de los problemas relajados no son de mayor interés respecto de lo que ya apreciamos en instancias anteriores, por lo que serán omitidos. Cabe destacar que la solución del problema relajado para el caso del **STSP-SCF** arroja un *Lower Bound* de 109, lo cual no está mal en términos de *gap* inicial.

Para ilustrar de forma acusada el buen rendimiento de **STSP-SCF** para problemas de gran tamaño, hemos realizado una última prueba con una instancia generada de 800 nodos, con una probabilidad de arista por nodo de 0.005 y una probabilidad de nodo requerido de 0.01. Encontramos el resumen de la ejecución en la Tabla 12. En este caso, no podemos representar gráficamente la solución por el tamaño del grafo, pero notamos que, con la formulación **STSP-TS** ni siquiera se obtiene una solución óptima en los 5 minutos de tiempo límite que tiene el solver; mientras que **STSP-SCF** la encuentra en tan solo 4 segundos.

Como nota final, podemos apreciar que para problemas cada vez mayores es posible que el *Lower Bound* de la relajación de la formulación basada en flujo *Single-Commodity* empiece a sufrir un poco. En esta instancia, el valor de dicha cota es de 91, bastante más alejada del óptimo que en anteriores instancias, tanto en términos absolutos como desde el punto de vista del *gap*.

Formulación	n° Variables	n° Restricciones	<i>Lower Bound</i>	<i>Gap</i>	Tiempo
STSP-SCF	9 632	6 424	153.00	0.0000	4.672 seg.

Tabla 12: Resultados de la quinta instancia del problema de Steiner con 300s de tiempo límite para Gurobi.

4. Conclusiones

En este trabajo hemos estudiado el comportamiento en términos de computación de diferentes formulaciones del *problema del viajante*, centrándonos en los tiempos de ejecución de cada uno, sus tamaños (tanto en variables como restricciones), su comportamiento en la primera iteración de *Branch and Bound* y la evolución de las cotas a lo largo de la resolución. Luego quisimos ir un paso más allá y realizar un estudio similar para una generalización del problema, denominado el *problema de Steiner*.

En la presentación de las formulaciones establecimos un orden teórico en el que se dispondrían las *Lower Bounds* de la relajación de cada formulación: fuimos capaces de confirmar en la práctica el orden dado para la formulación del *problema del viajante*, pero en el *Problema de Steiner* nos encontramos con que la formulación *Time-Staged* se encontraba sistemáticamente por debajo, a nivel de *Lower Bound*, respecto a la formulación basada en flujo *Single-Commodity*. Es un resultado relevante porque en la literatura consultada para la realización de este trabajo [1] se conjeturaba que la cota inferior de la relajación de *Time-Staged* se encontraría siempre entre la obtenida con la relajación de las otras dos. Además, conseguimos ver también cómo la cota inferior de **STSP-SCF** se acercaba bastante a la dada por la formulación mucho más fuerte de Fleischmann.

También, pudimos observar cómo todas las formulaciones daban problemas al aumentar el tamaño, salvo las basadas en flujo *Single-Commodity*, tanto para el *problema del viajante* como para el *problema de Steiner*. En ambos casos no solo vimos que presentan tiempos de ejecución muy reducidos, sino que además estos tiempos no escalan demasiado, quedando consistentemente por debajo de los 5 segundos para cualquier instancia de las probadas (hasta un máximo de 100 nodos para **TSP** y 800 nodos para **STSP**). Como ya vimos en el trabajo, lo más seguro es que realmente esta formulación necesite de más iteraciones que las demás, pero al tener un crecimiento del número de variables y de restricciones tan bajo, cada subproblema se resuelve en poco tiempo, compensando así el tiempo global de ejecución.

Apreciamos también problemas derivados del tamaño en el resto de formulaciones, sobre todo por el tiempo dedicado a la preparación del problema, antes de la llamada al *solver*. Tanto **TSP-DFJ** como **STSP-F** consumen demasiado tiempo en la lectura del problema, dejando de ser útiles alrededor de los 25 nodos.

Siguiendo lo anterior, **TSP-MCF**, aunque no crece tanto como las anteriores, al aumentar el tamaño se ve lastrada por las tareas de preparación y algoritmos previos que utiliza el *solver*, etapa en la que ha llegado a pasar hasta 4 minutos. Como ya vimos, parece que lo que más problemas causa en esta línea es el aumento en el número de restricciones —por lo menos en lo relativo a los tiempos de preparación— pues la formulación *Time-Staged* del *problema del viajante* cuenta con el mismo número de variables, pero no se observa la misma problemática.

Referencias

- [1] A. N. Letchford, S. D. Nasiri, and D. O. Theis. Compact formulations of the steiner traveling salesman problem and related problems. *European Journal of Operational Research*, 228(1):83–92, 2013.
- [2] S. Vajda. *Mathematical Programming*. Addison-Wesley, 1961.